

Grand Java Hotel Management System

Project Report

Build Your Own Project — VITyarthi

Subject: Java Programming (CSE2006)

Name: Tanishq Chaudhary Reg No: 25BAI10574

Slot: B21+E14

Introduction

The Grand Java Hotel Management System is a modular console application built to bring some of the everyday front-desk work of a hotel onto a computer instead of a paper register. It looks after rooms, guests, reservations, cancellations, billing, and a simple dashboard, and along the way it puts several core Java programming ideas to practical use.

1. Problem Statement

Hotels that still keep their records by hand run into the same problems again and again: rooms get double-booked, the status of a room falls out of date, staff end up retyping the same guest details more than once, and bills are worked out by hand with all the room for error that involves. This project sets out a structured workflow instead — one place to store guest details, check which rooms are free, create a reservation, and generate the resulting bill.

2. Objectives

The project was built with a few clear goals in mind. It needed to put Java classes, objects, constructors, and methods to real use, and to show encapsulation and modular design working in practice rather than just in theory. Records are held in memory using collections while the program runs, and file handling takes care of persisting them to CSV storage between sessions. User input is validated and common errors are handled gracefully, with the overall aim of producing something that is both a practical piece of software and a demonstrable academic project.

3. Functional Requirements

The system goes beyond the minimum of three functional modules that were required, and instead covers six areas of hotel operation. Room management displays the hotel's rooms and keeps track of which ones are available. Guest management handles registering new guests and looking them up again later. Booking management covers creating a reservation and cancelling one when plans

change. Billing works out the room charge, any service charges, and the applicable tax. Reporting shows room counts and the current occupancy rate, and persistence saves every record to CSV files and reloads them the next time the program runs.

4. Non-Functional Requirements

Beyond what the system does, some thought went into how well it does it. Menu operations are expected to complete quickly given the scale of data an academic project deals with, and the interface leans on numbered menus and clear prompts to stay usable. Invalid numeric input is caught and the user is asked to try again rather than the program crashing outright. On the code side, responsibilities are split between model and manager classes to keep things maintainable, CSV storage keeps resource use light by avoiding a full database server, and file and input exceptions are caught and reported with messages a user can actually understand.

5. System Architecture

The application follows a straightforward layered architecture: the console interface talks to manager classes, which operate on model classes, which are in turn persisted through a FileManager to CSV files. Main is responsible for coordinating the user's interaction with the program, the manager classes hold the actual business logic, the model classes represent the data itself, and FileManager centralises everything to do with reading and writing files.

6. Design Diagrams

The repository includes Mermaid source files covering the architecture, use-case, workflow, sequence, and class diagrams, which can be rendered directly in GitHub or any Mermaid-compatible editor. An entity-relationship diagram wasn't needed here, since the project deliberately stores its data in files rather than a relational database.

7. Design Decisions and Rationale

CSV files were chosen for persistence so the project can run on a standard JDK install without anyone needing to set up a database server first. Splitting the logic into separate manager classes makes the codebase easier to read and maintain, and keeping InputValidator on its own means validation logic never gets tangled up with business logic. For the billing demonstration, a flat 12% tax rate is applied consistently throughout.

8. Implementation Details

The project is made up of thirteen Java classes in total. Room, Guest, Booking, Bill, and Staff are the model classes that represent the hotel's data. RoomManager, GuestManager, BookingManager, and

BillManager carry out the corresponding domain operations. FileManager takes care of reading and writing the CSV files, ReportManager puts together the dashboard information, InputValidator checks interactive input before it's used, and Main drives the overall console workflow.

9. Screenshots and Results

Running the application produces a main menu, a list of rooms, guest records, booking confirmations, cancellation messages, generated bills, and dashboard statistics. The repository can be run directly to capture real screenshots for the final version of the report.

10. Testing Approach

Testing was built around functional test cases covering valid operations, invalid menu selections, invalid numeric input, attempts to book an already-occupied room, booking cancellation, billing calculations, and date validation. The complete test table is kept separately in docs/TestCases.md.

11. Challenges Faced

A few implementation challenges came up along the way. Keeping room availability in sync with booking creation and cancellation took some care, as did making sure generated IDs stayed unique within a single run. Persisting records correctly after every change, and validating interactive console input without ever causing the program to terminate unexpectedly, were the two that took the most iteration to get right.

12. Learnings and Key Takeaways

Working through this project reinforced a lot of core object-oriented ideas: encapsulation, constructors, methods, collections, file handling, exception handling, control flow, and modular programming in general. It also introduced some of the wider software engineering practices that come with building something end to end, from requirements analysis and diagrams through to testing, documentation, and version control with Git.

13. Future Enhancements

There's a clear path for taking this further. Login and role-based access would be a natural next step, alongside a proper graphical interface built with JavaFX or Swing in place of the console. Moving from CSV files to a MySQL database with JDBC would let the system handle real-world data volumes, and support for multiple-room or multi-guest reservations, along with date-overlap validation for future bookings, would make the booking logic more robust. Further out, payment modes with printable invoices, more advanced revenue and occupancy analytics, and a proper suite of JUnit unit tests would round the project out.

14. References

This project draws on the Build Your Own Project submission guidelines supplied for the course through VITyarthi, along with the standard Java SE documentation and general Java library concepts used throughout the implementation.

Appendix A — Run Instructions

```
javac -d out src/hotelmanagement/*.java  
java -cp out hotelmanagement.Main
```

Appendix B — Suggested Demonstration Flow

A demonstration works well as a simple walkthrough: start by displaying the available rooms, then add a guest and create a booking for them. Show the room now marked as occupied, generate the bill for that booking, and check the dashboard to see the occupancy figures update. Finally, cancel the booking, confirm the room shows as available again, and exit the program.